Concesionaria de autos

Documento canónico de alcance del miniproyecto · Versión 1.0 · 25 de octubre de 2026

Decisión de producto. Sistema web de uso exclusivamente administrativo, para una sucursal, que registra clientes, proveedores de vehículos, unidades, ventas administrativas y revisiones. El objetivo adicional es probar autenticación administrada y autorización en React + FastAPI antes de tomar decisiones para DaVinciMusic.

Fuentes de diseño: requerimientos y sprints de DaVinciMusic (RF-01 a RF-43; RNF-01 a RNF-24), diagramas y diccionario de datos de la concesionaria. La decisión posterior de seis tablas sustituye la propuesta preliminar de tres.

1. Límites y principios

• Una sola sucursal; sin multiempresa, ubicaciones múltiples ni traspasos entre sucursales.

• Los clientes y proveedores son registros comerciales; no inician sesión ni disponen de portales.

• Cada coche procede de un proveedor; se registra su venta a un solo comprador por operación.

• Solo una venta vigente por coche. Se preservan cancelaciones; no hay recompra ni reventa de la misma unidad.

• Todos los importes se expresan en MXN. Una venta es un registro, no un cobro electrónico.

• Revisiones de vehículos de la concesionaria, realizadas antes o después de la venta; no se reciben coches externos.

2. Arquitectura y tecnologías

Capa	Decisión / responsabilidad
Frontend	React + Vite; formularios, navegación, tablas y control de sesión.
API	Python + FastAPI; validación, reglas de negocio, permisos y respuestas HTTP/JSON.
Persistencia	PostgreSQL (Neon es opción de alojamiento); SQLModel/SQLAlchemy como ORM.
Identidad	Clerk propuesto para la prueba; FastAPI verifica el token y consulta autorización local.
Documentación	SQL canónico, diagramas .mmd/.png, diccionario, README y OpenAPI /docs.

3. Funcionalidades incluidas (RF)

ID	Módulo	Requerimiento verificable
RF-01	Acceso	Iniciar y cerrar sesión con proveedor externo; impedir llamadas anónimas a la API privada.
RF-02	Roles	Autorizar por rol y por estado activo del usuario interno; administración de usuarios restringida.
RF-03	Clientes	Alta, listado, detalle, edición y desactivación; búsqueda sencilla.
RF-04	Proveedores	Alta, listado, detalle, edición y desactivación de proveedores de vehículos.
RF-05	Vehículos	Alta, listado, detalle y edición; proveedor obligatorio, serie única y matrícula opcional.
RF-06	Ventas	Registrar una venta de un coche a un cliente; consultar historial y cancelar con motivo y responsable.
RF-07	Revisiones	Registrar, listar por coche y editar de forma controlada; responsable y kilometraje.
RF-08	Interfaz	Menú interno, tablas, formularios, confirmaciones, estados vacíos y errores claros.
RF-09	Consultas	Mostrar inventario y estado de venta derivado; historial de ventas y revisiones por unidad.

4. Roles y permisos

Rol	Permitido	Restricciones
Administrador	Todos los módulos; altas/bajas lógicas de usuarios y asignación de roles.	No puede omitir integridad ni borrar historial.
Gerente	Clientes, proveedores, coches, ventas y revisiones.	No administra usuarios ni roles.
Mecánico	Datos técnicos de coches, historial de revisiones y revisiones propias.	Sin precios, ventas, contactos comerciales ni cambios de roles.

La interfaz puede ocultar acciones, pero FastAPI valida todos los permisos. Un token válido no autoriza automáticamente: se exige una identidad externa vinculada a un usuario_interno activo.

5. Modelo de datos canónico

Tabla	Responsabilidad y relaciones
cliente	Comprador histórico de venta; contacto y estado activo.
proveedor	Origen obligatorio de coche; contacto y estado activo.
coche	Unidad identificada por id interno y serie única; proveedor, datos físicos y precio de referencia.
venta	Un coche, un cliente, registrador, fecha e importe; cancelación con usuario, momento y motivo.
revision	Coche, responsable, fecha, kilometraje, tres indicadores de cambio y observaciones.
usuario_interno	Identificador externo único, nombre, rol y estado activo; sin contraseñas.

Las seis tablas están conectadas. El diseño evita duplicar el comprador o el indicador vendido dentro de coche. Las ventas canceladas permanecen en el historial, pero un índice parcial impide dos ventas registradas para la misma unidad.

6. Reglas de integridad y operación

• PK autogeneradas, FK restrictivas, unicidad de identidad externa y serie, matrícula única si existe, campos obligatorios y dominios válidos.

• Importes mayores que cero; kilometraje no negativo; estados de venta: registrada o cancelada.

• Una cancelación requiere autor, fecha y motivo. La corrección de una venta se realiza cancelando y registrando otra; no se sobrescribe el histórico.

• La API impide fechas futuras para venta y revisión, valida año modelo hasta año actual + 1 y el formato de correo.

• Nuevos coches requieren proveedor activo; nuevas ventas requieren cliente activo y unidad sin venta vigente.

• Usuarios y contactos con historial se desactivan. No se borran ventas ni revisiones históricas en operaciones ordinarias.

• La sesión determina el usuario registrador/cancelador. La revisión puede tener un responsable técnico distinto del capturista, autorizado por el rol.

7. Requerimientos no funcionales (RNF)

ID	Criterio
RNF-01	Separar frontend, API, modelos y persistencia; HTTP/JSON y CORS restringido.
RNF-02	Persistencia real en PostgreSQL; restricciones en BD y validaciones complementarias en API.
RNF-03	Autenticación segura, autorización por mínimo privilegio y secretos fuera de Git.
RNF-04	Interfaz responsive básica, etiquetas, contraste, teclado y errores comprensibles.
RNF-05	Pruebas de CRUD, integridad, roles, token ausente/inválido y persistencia tras recarga.
RNF-06	Código modular, documentación de instalación y configuración de ejemplo sin secretos.
RNF-07	Control de errores sin trazas ni credenciales expuestas al usuario.

8. Exclusiones explícitas

No incluye tienda pública, carrito, pagos, facturación, reembolsos, métodos de cobro, financiamiento, cupones, órdenes de compra o abastecimiento, refacciones, servicios detallados, órdenes de taller, inventario de piezas, portales de clientes/proveedores, cuentas de proveedores, múltiples sucursales, reservas, dashboards complejos, reportes Excel, notificaciones de correo propias ni despliegue comercial de producción.

9. Criterios de aceptación del MVP completo

• Un administrador autorizado entra y sale; sin sesión o con token inválido, la API privada responde con rechazo.

• Un usuario autenticado no registrado o inactivo en usuario_interno no consigue acceso.

• Gerente puede registrar proveedor, coche y cliente; registrar una venta y consultar su historial.

• Una segunda venta vigente del mismo coche se rechaza, incluso ante solicitudes concurrentes.

• La cancelación conserva sus datos completos y permite registrar una nueva venta, manteniendo el historial.

• Mecánico puede registrar su revisión y consultar el historial técnico, pero no obtener campos financieros o comerciales.

• FK inválidas, campos vacíos, importes no positivos y otros datos inválidos generan respuestas claras.

• Los cambios válidos persisten al refrescar la interfaz y se reflejan en PostgreSQL.

• El usuario puede navegar entre secciones y completar los flujos sin ejecutar SQL manualmente.

10. Secuencia sugerida de implementación

Fase	Entrega demostrable
1. Base	Estructura modular FastAPI, configuración, SQL canónico y conexión a PostgreSQL de pruebas.
2. Núcleo	Modelos, esquemas, CRUD de clientes, proveedores, coches y revisiones; pruebas.
3. Identidad	Clerk y autorización local por rol; rutas protegidas en frontend y backend.
4. Ventas	Registro/cancelación transaccionales y consultas de historial.
5. Interfaz	React/Vite conectado a la API, validaciones y estados de carga/error.
6. Cierre	Pruebas de permisos, concurrencia, persistencia y README de instalación.

11. Decisiones de implementación y advertencias

• El SQL adjunto crea tablas en una base nueva; no se ejecuta sobre la base académica de tres u ocho tablas ni migra sus datos.

• Clerk y Neon son opciones técnicas propuestas. Su configuración final dependerá de cuentas y credenciales de prueba.

• El diccionario y los diagramas proporcionados establecen el diseño lógico; el SQL es la fuente ejecutable de esquema.

• Las reglas de roles, fechas relativas y estados activos entre tablas exigen lógica del backend; los CHECK no pueden sustituirla.

• El objetivo es un miniproyecto de aprendizaje. Este alcance no implica seguridad, auditoría y operación certificadas para producción.